

Relationships between objects

Gregorics Tibor and Pintér Balázs

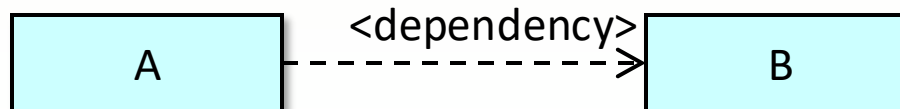
Types of object relationships

- ❑ When objects communicate with each other (by calling each other's methods synchronously or asynchronously, sending signals to one another, or directly manipulating each other's data members), or when they are similar to each other, a relationship forms between them.
- ❑ Most of the type, at the implementation level, **one object is an attribute of the other object** (association, aggregation, and composition).
- ❑ The connection can be:
 - **Dependency**
 - **Association**
 - **Aggregation** (part-of)
 - **Composition** (stricter part-of)
 - **Inheritance**

Dependency

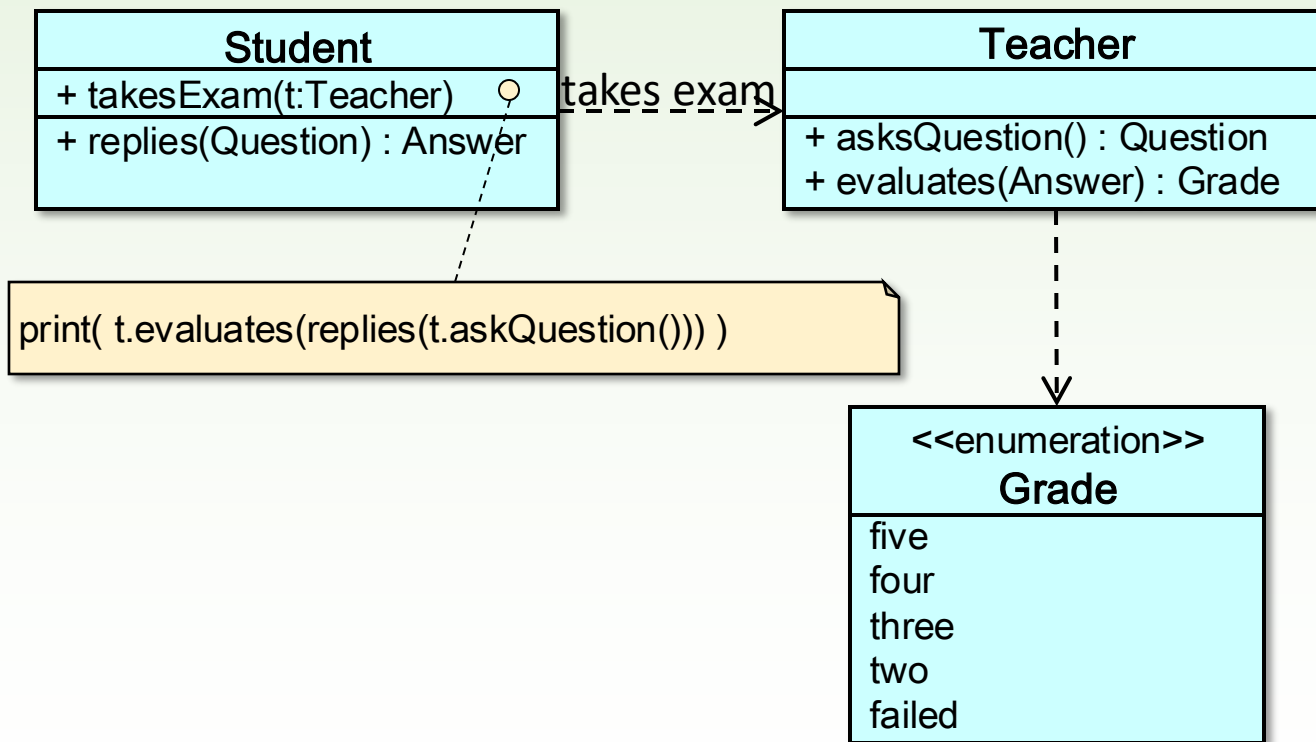
□ Class A depends on class B, if

- The two classes temporarily (for a short time) establish a connection in such a way that an object of class A either receives an object of class B as a **parameter** in one of its methods or instantiates an object of class B **locally** for the purpose of
 - calling one of its methods, or
 - sending a signal, or
 - passing on its reference (as a return value or embedded in an exception throw).
- One of the methods of class A calls a **class-level method** of class B. (This is not a relationship between objects.)
- An object of class A **uses an object of class B**, which in class A could be:
 - the value of the data member
 - a method's return value



Examples of dependencies

- A student is taking an exam with a teacher, who asks them a question and then evaluates their answer.



Relationships between objects

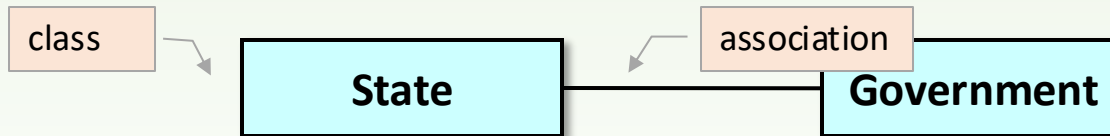
1st part

Association

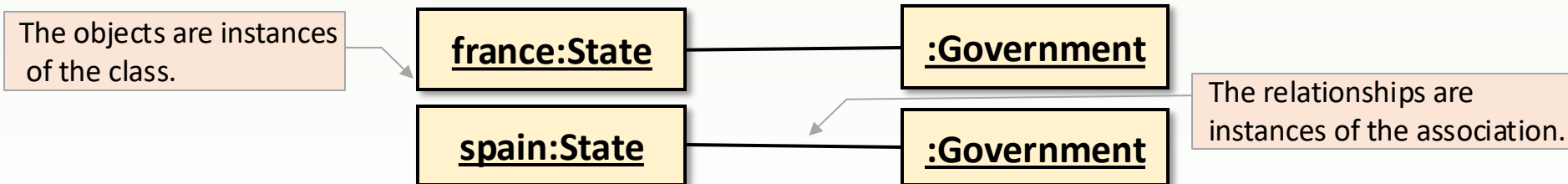
Gregorics Tibor and Pintér Balázs

Association

- ❑ The association is a **long-term relationship** between objects, during which the objects continuously send messages to each other.
- ❑ In mathematical terms, an association is a relation on the direct product of sets containing instances (objects) of the given classes:
 - An association describes multiple object relationships, and the same object can appear in multiple relationships within an association, but it can also appear in the relationships of different associations.



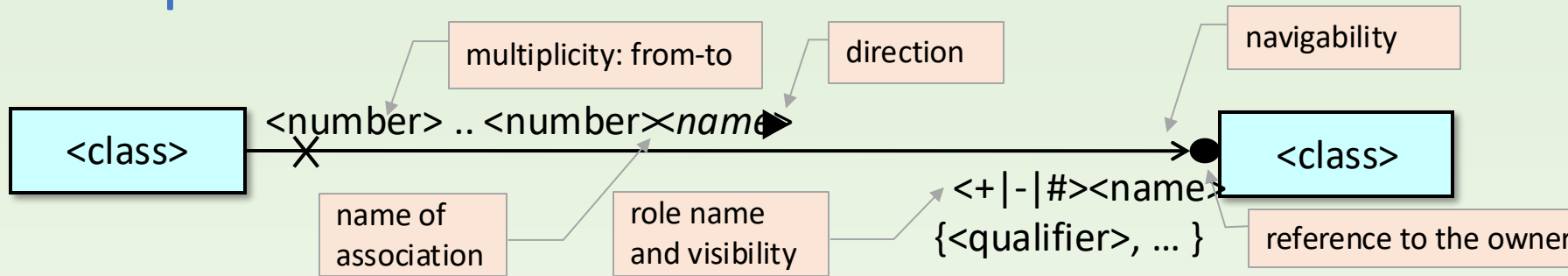
A possible instantiation (population) of the class diagram:



4

The hallmark of object-oriented languages is **conceptual-level abstraction**. The class serves as an abstract description of objects, and the association serves as an abstract description of relationships.

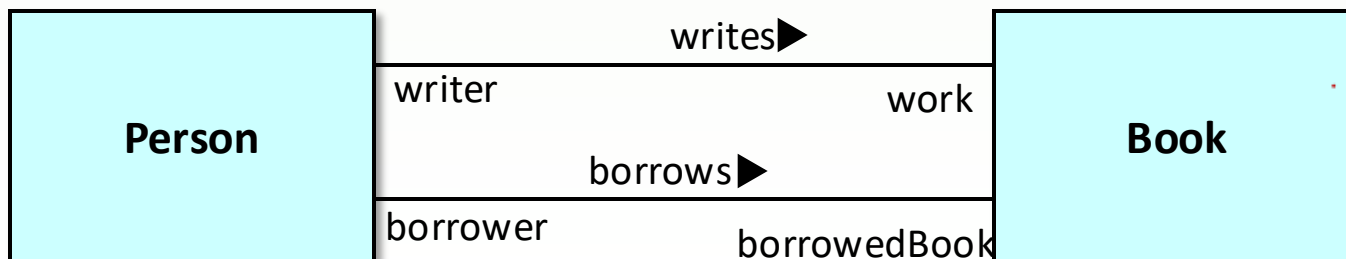
Properties of association



- ❑ The properties of the association characterize the relationships it describes:
- **name**: the name of the relationship
 - **direction (of reading)**: the conceptual relationship between the objects
 - **multiplicity**: the number of objects that can be connected to an object.
 - **arity**: the number of different classes participating in the relationship
 - **navigability**: which object can access which object directly
 - **role names**: the names of the objects or collections in the relationship
 - **visibility** of the role
 - **owner** of the role
 - **qualifier** of the collection for this role name

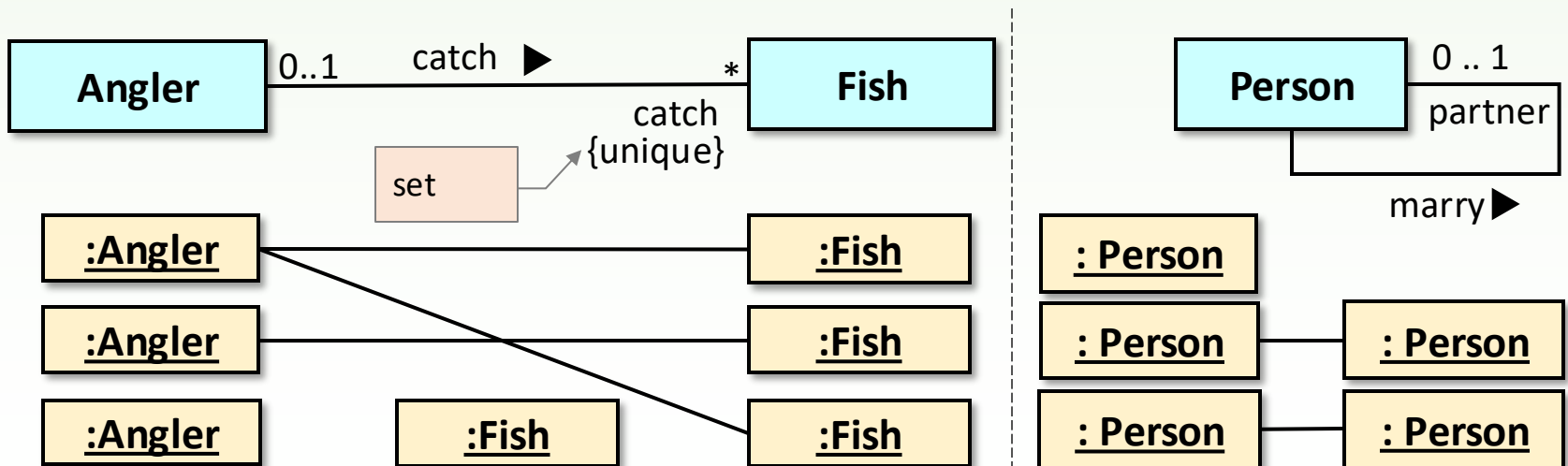
Name and direction

- ❑ There is a close relationship between the associations of an object-oriented model and the sentences in the textual description of the problem to be solved. Based on this:
 - The **name of the association** is usually the predicate of the sentence, which is often also the name of a method of one of the related classes.
 - The remaining words of the sentence (like the subject or object) provide the names of association ends (**role names**), which serve to allow the related objects to reference each other.
- ❑ The tip of the black triangle drawn next to the name of **binary** associations indicates the **direction** of the association's effect.
 - It points from the subject of the sentence of association towards the other direction (which is often the object of the sentence).



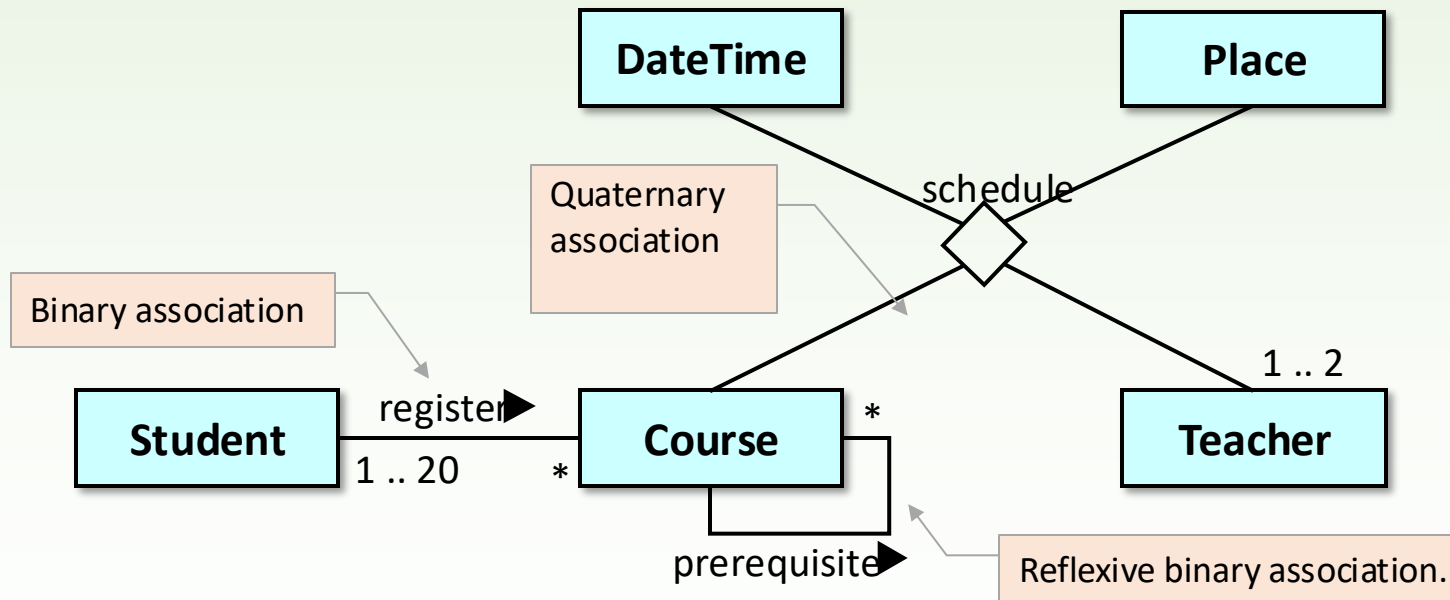
Multiplicity

- The multiplicity of an association indicates **how many other objects one object can be associated with at the same time** through that association. This can be a natural number (which defaults to 1), or a range of natural numbers expressed as min..max. If an asterisk * is used instead of a number for the multiplicity, it means that value can be any number.
- You can specify that the objects associated with a given object
 - should be all **unique** {unique}
 - Should be **ordered** {ordered}



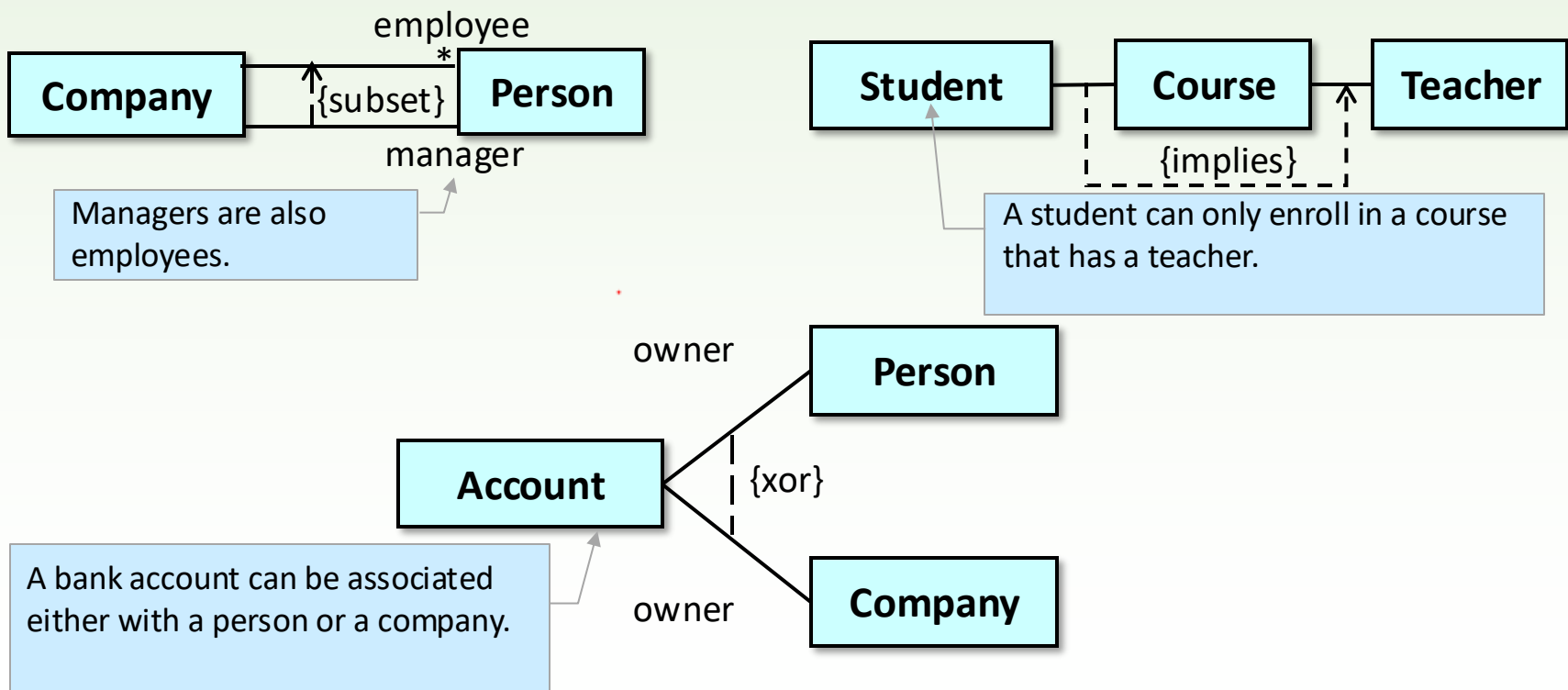
Arity

- The arity of an association refers to how many classes that single association links together. So far, we've only seen examples of binary associations, where a connection is always created between two objects.



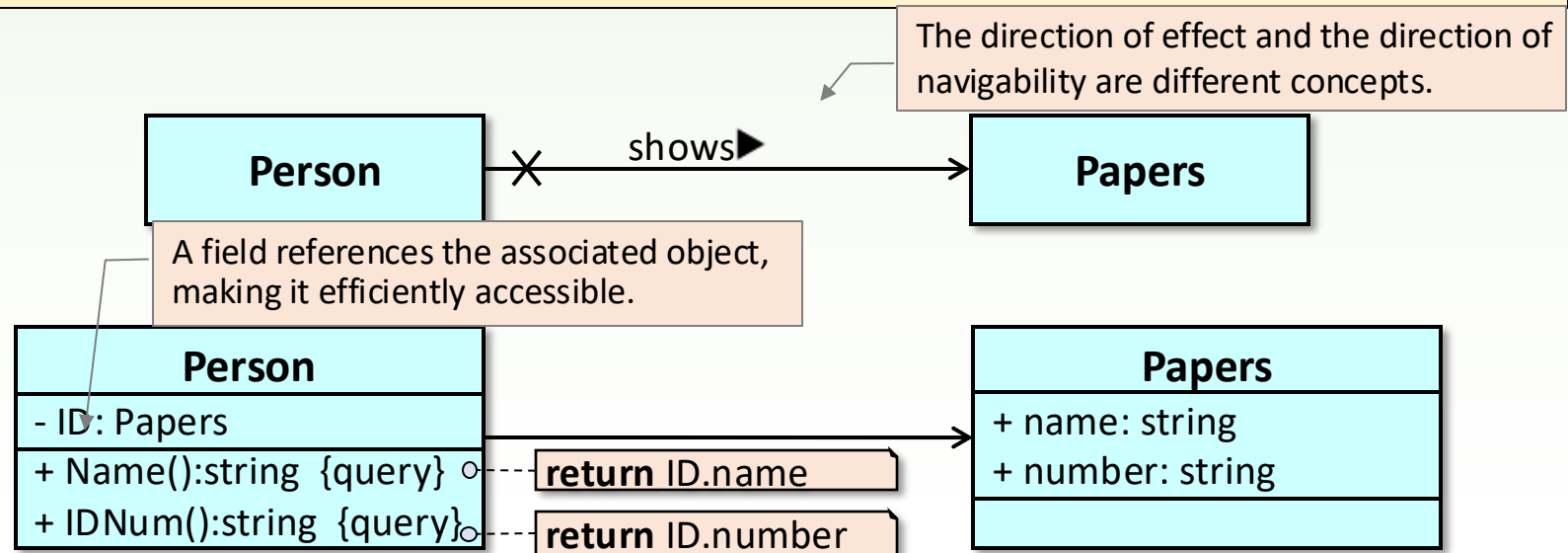
Constraints on associations

- We can specify logical conditions between associations (subset, and, or, xor, implies, etc.) that express **constraints between the relationships** of the given associations.



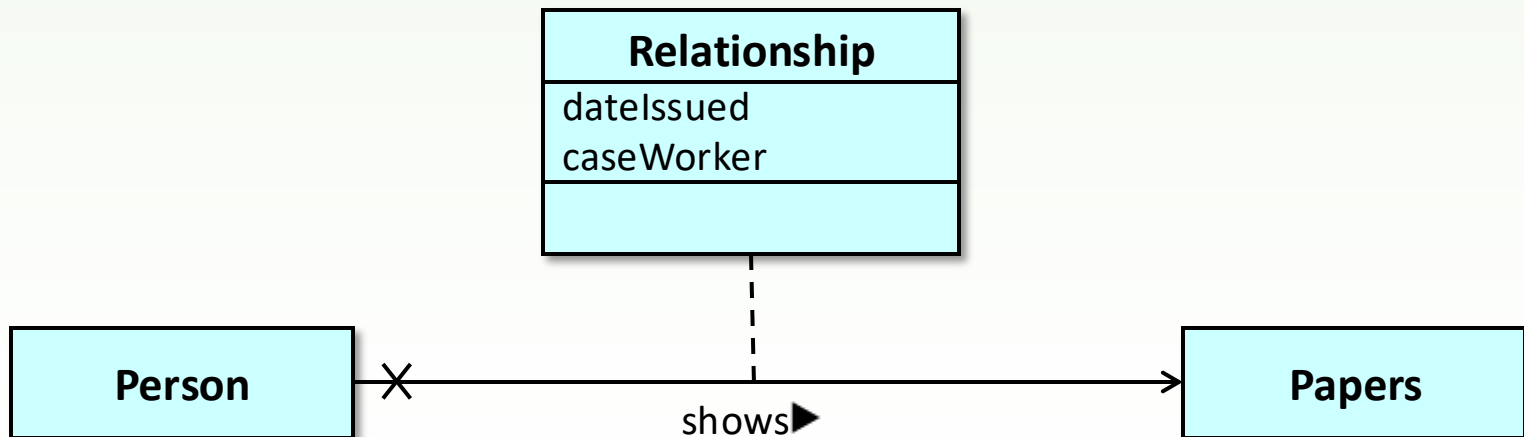
Navigability

- In an association, navigability shows the object(s) of which class should be efficiently accessible from which other object(s).
 - The **arrow points to** the class of the object **to be reached efficiently**.
 - The "x" symbol at the end of an association is next to the class whose object **doesn't need to be efficiently accessed**.
 - An unmarked association end indicates that it **hasn't been decided yet** whether navigability should be supported there.



Association class

- We can think of an association relationship as an independent entity that stores references to the connected objects. Plus, it can have other data members (and even methods). An association class is used to describe such relationships.
- However, most programming languages don't treat relationships as independent entities, and they don't allocate separate memory space for them. They don't even have a concept of an association class. This makes implementing it a real challenge.



Relationships between objects

2nd part

Modeling the Implementation of Associations

Gregorics Tibor and Pintér Balázs

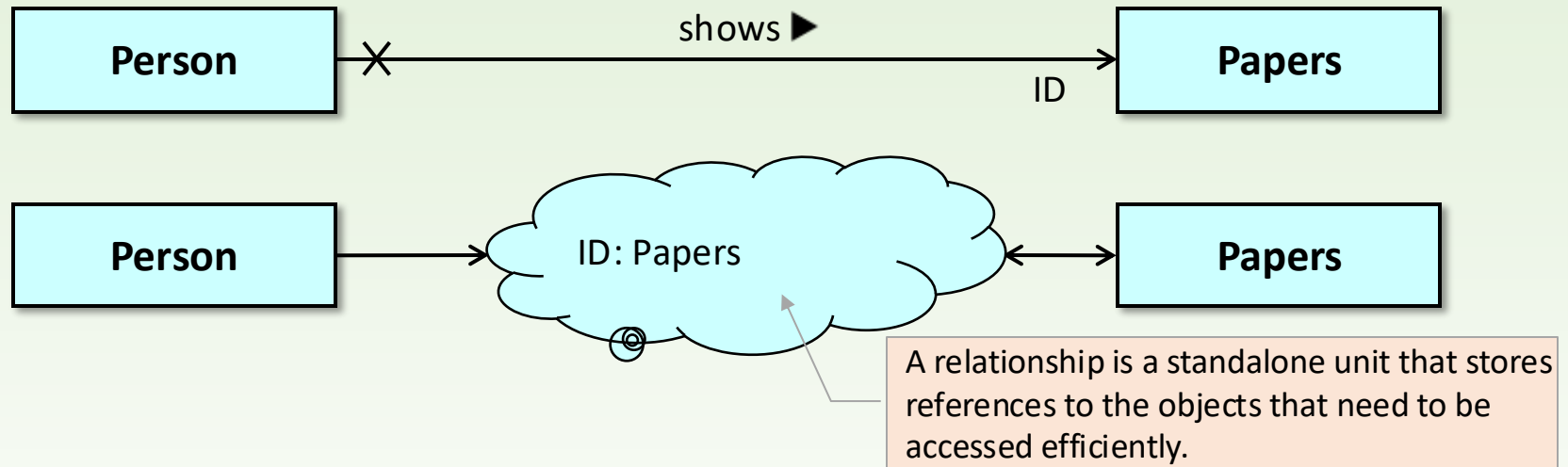
Ownership of role names

- We've seen that introducing role names is useful in the concept-building process of modeling. On the other hand, we can use them to refer to objects within a relationship.
- If you need to efficiently navigate to an object within a relationship, you have to know the object's reference. We store this identifying it with a specific role name. But where? Who should own the role name?
 - **the relationship** itself, assuming that the relationship has its own independent storage space it can access by default. In this case, the role name appears as a data member of the association class that describes the association.
 - **the object** (or objects) from which the object designated by the role name needs to be efficiently accessed. In this case, the role name becomes a data member of that object.
- You can control the **visibility** of role names the same way you control the visibility of data members.

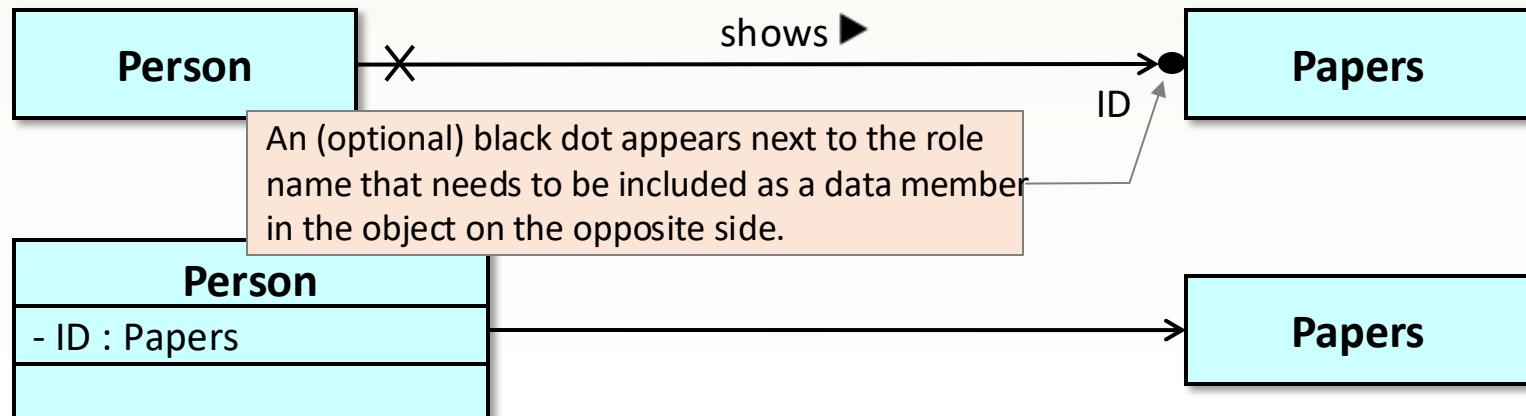
more common

Marking the owner of a role name

The relationship itself is the owner of the role name

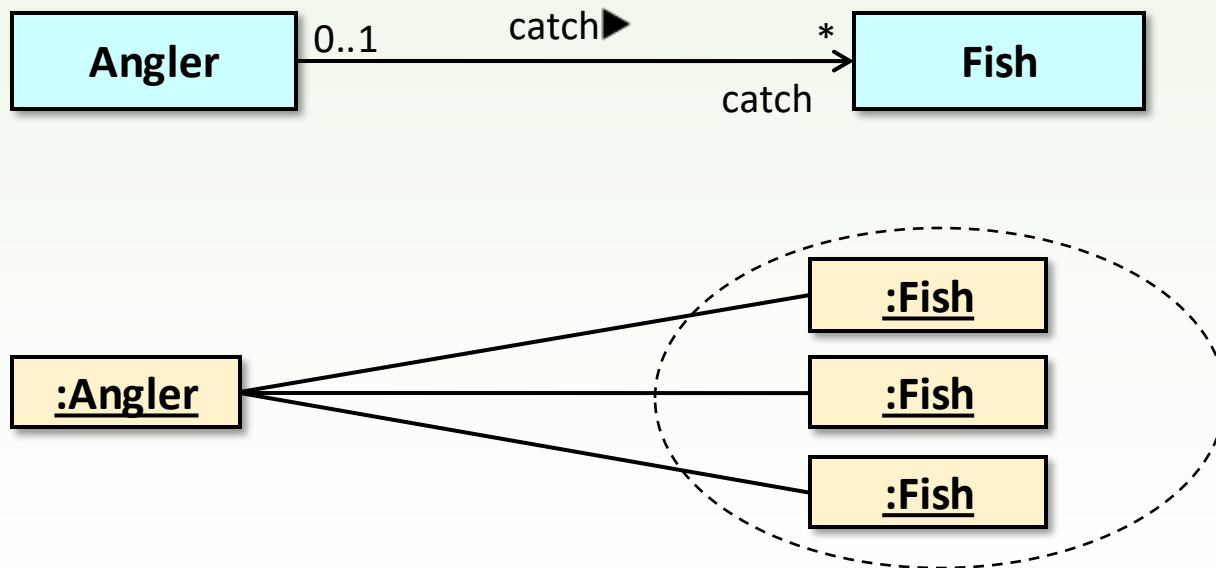


The other object in the relationship is the owner of the role name



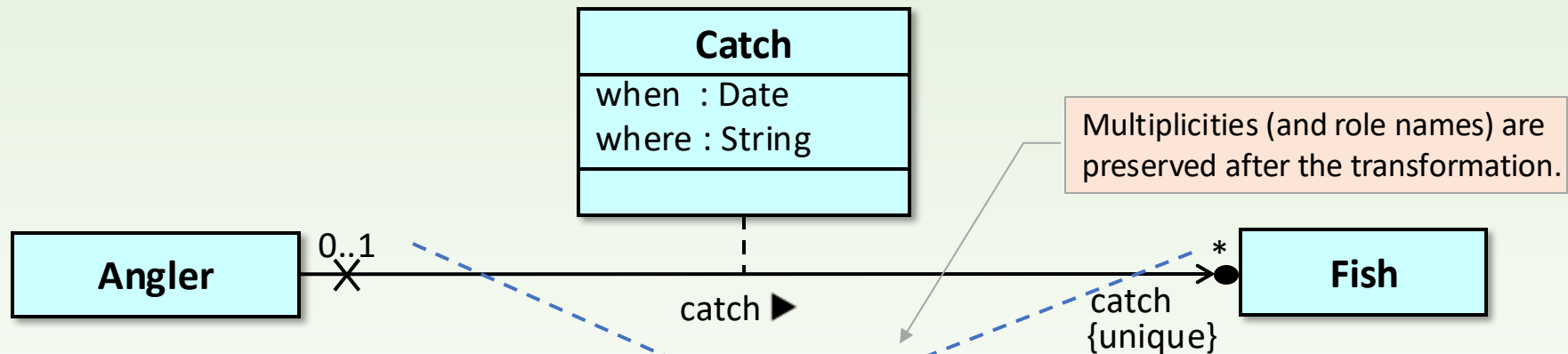
Role name representing collection

- When a multiplicity indicating "many" is next to one of the role names in an association, that role name identifies a collection of objects. In other words, that role name represents a set or group objects.

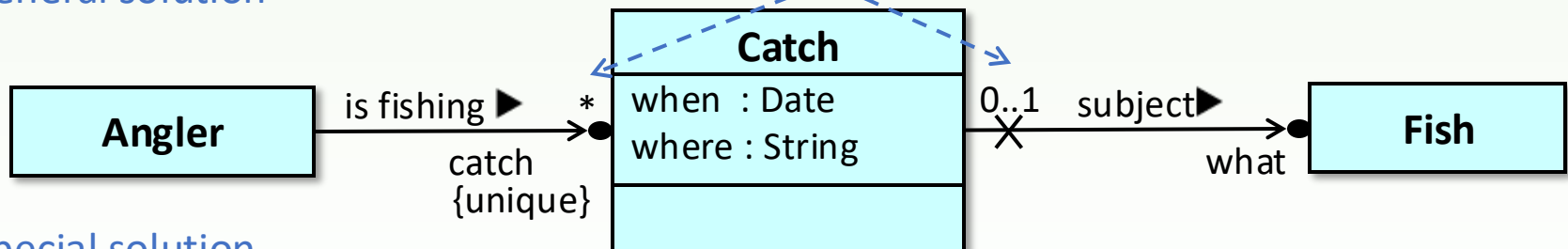


Eliminating Association Classes

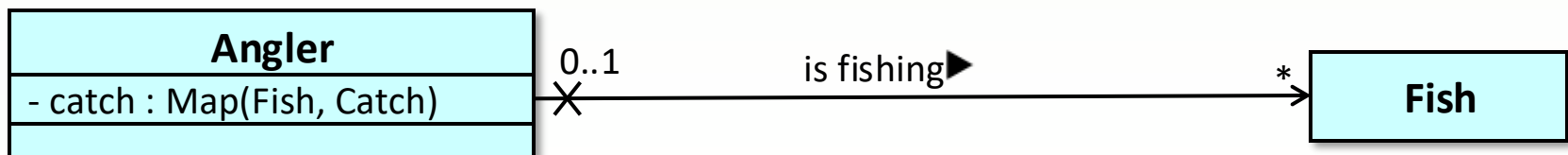
- If a programming language doesn't support association classes, then relationships won't have their own dedicated storage space, unless you replace the association class with a regular class.



General solution



Special solution

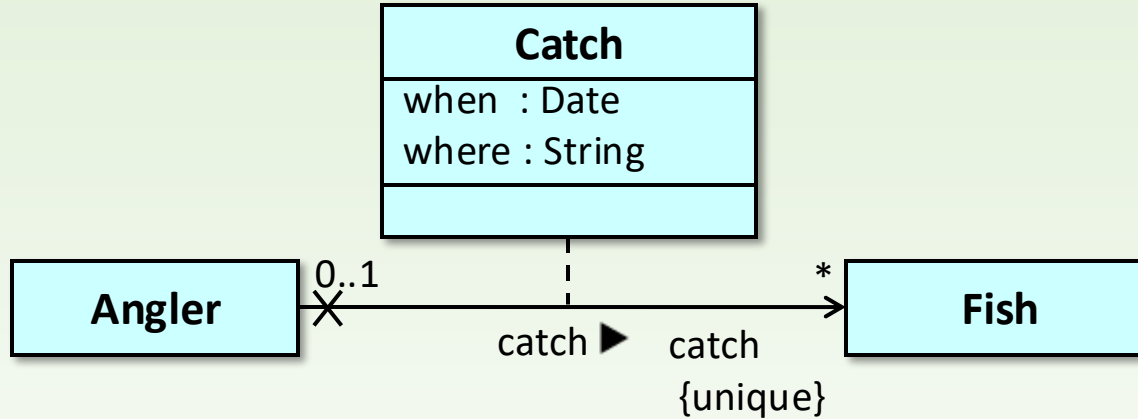


Modeling Levels

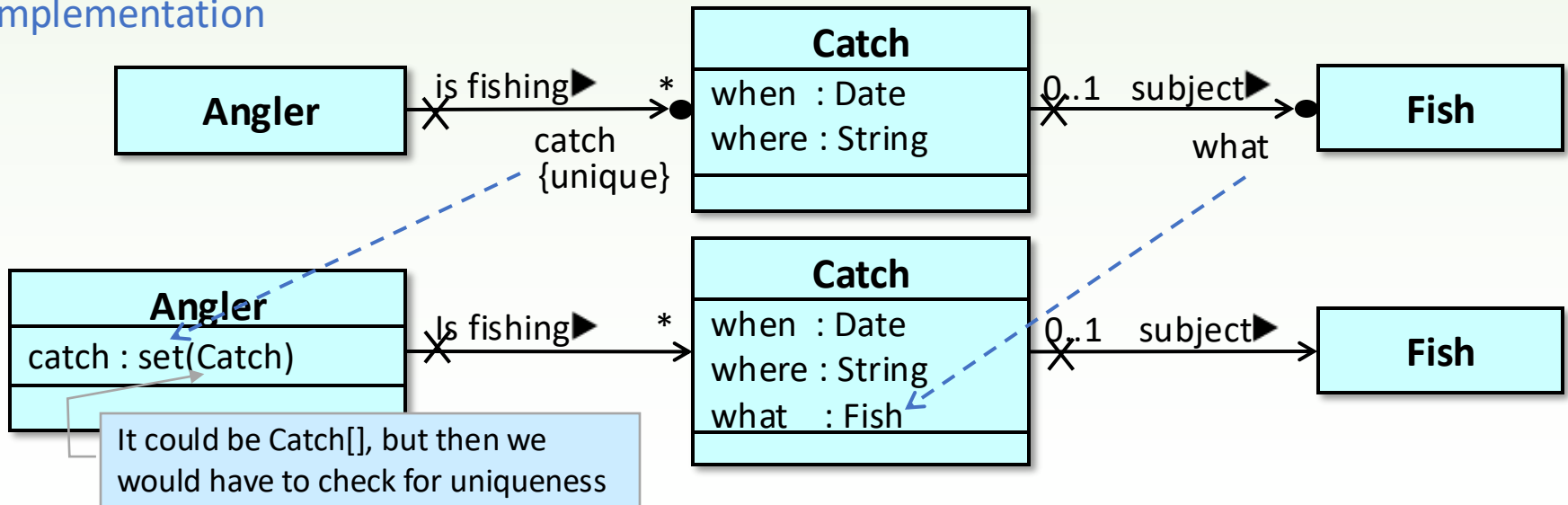
analysis



planning



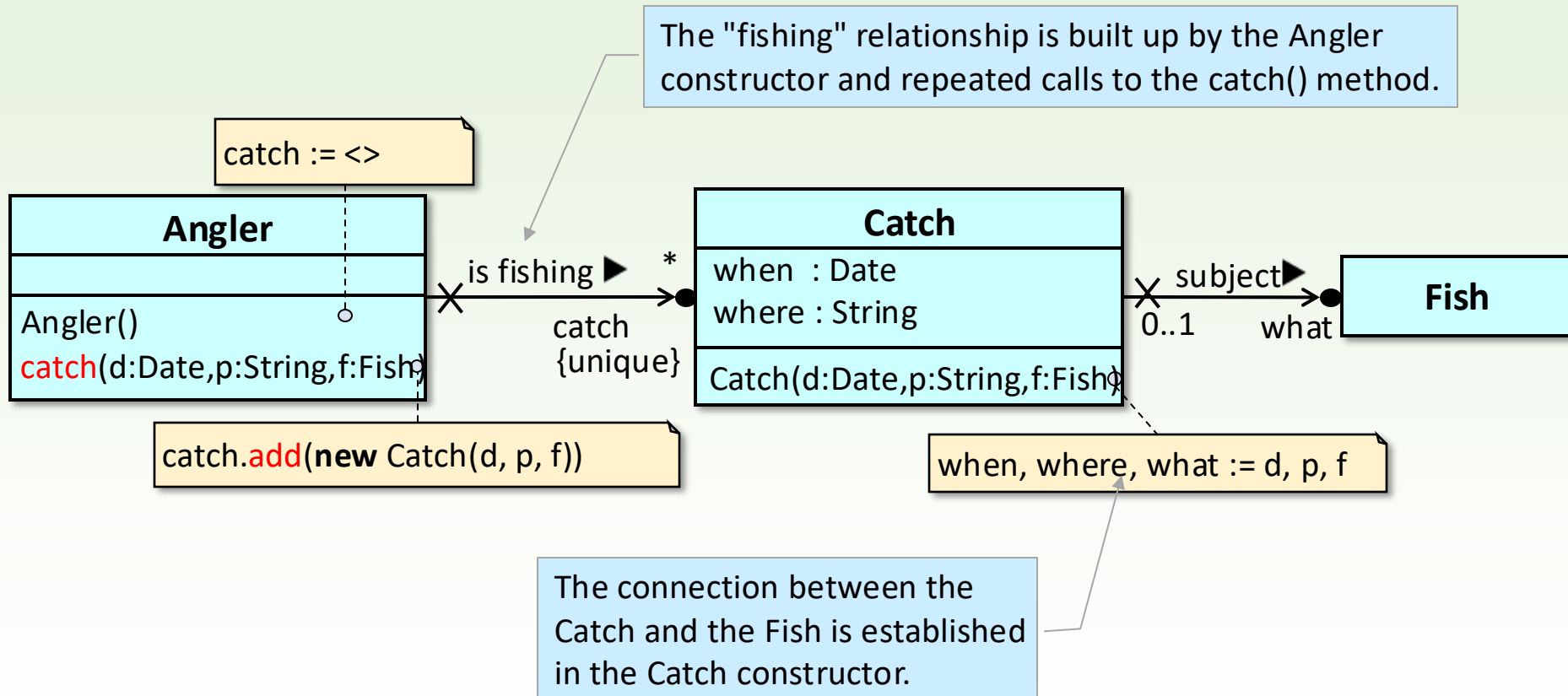
implementation



Establishing Association Relationships

- An association always has a corresponding method that establishes its relationships, and in many cases, that's a constructor.
- A method that establishes a relationship either creates the objects to be connected, or receives references to some or all of them as parameters, and assigns these to the appropriate role names.
- When establishing the relationship, you have to be careful about:
 - the multiplicities of the association
 - role name constraints (e.g., unique)
 - relationships to other associations (e.g., implies)

Example of establishing relationships



Relationships between objects

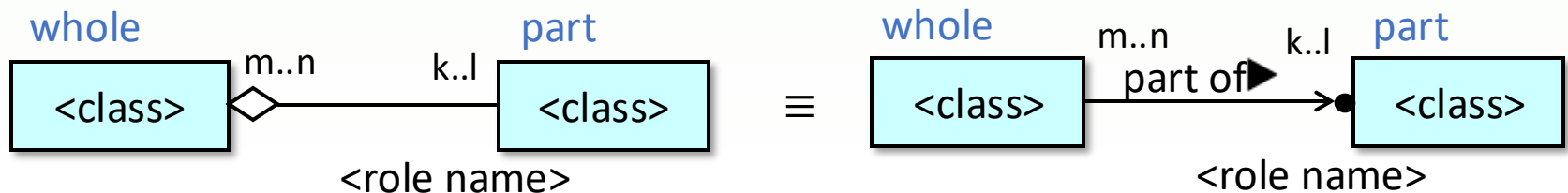
3rd part

Aggregation and composition

Gregorics Tibor and Pintér Balázs

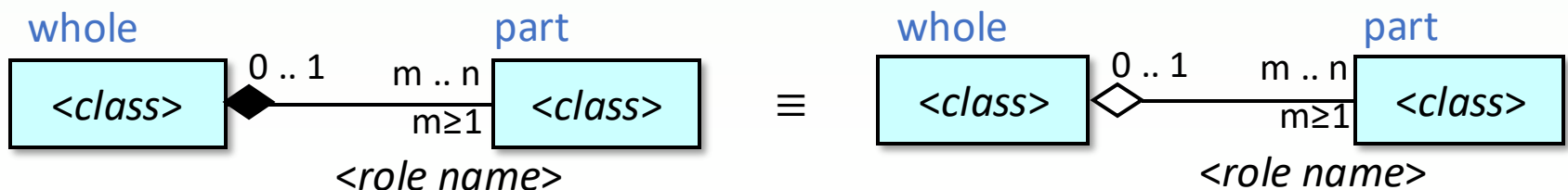
Aggregation

- **Whole-part relationship.** It is a **binary relationship** used to express that one object is part of another, where the relationships **cannot form a directed cycle** (an object cannot be its own part, even indirectly).
- It is possible that
 - a whole doesn't have a part
 - a whole exists on its own, without parts
 - a part is part of multiple wholes at once
- You can efficiently navigate the relationship towards the part object by having the whole object store the role name of its property.
- It's the responsibility of the "whole" object to establish the aggregation relationship. A method of the whole object, which could be its constructor, places the reference to the part object in the appropriate data member of the owner.

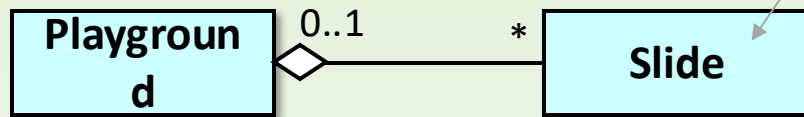


Composition

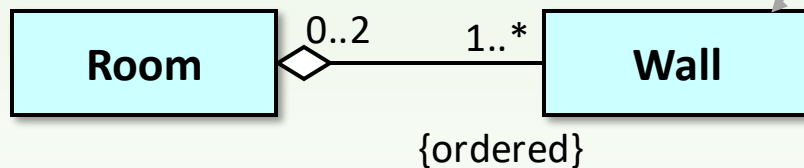
- Composition is a **stronger** whole-part relationship where
 - the whole **cannot exist without the part**,
 - the part can only be part of a **single object at once**.
- Many people also associate one of the following increasingly strict constraints with composition: the contained object (the part)
 - **always has container object (whole)**: (cannot exist on its own).
 - **its container object (whole) never changes**
 - **its lifetime is the same as its container object (whole)**
- Composition is established by the constructor of the container (whole) object. but – except for the last, most strict interpretation – other methods can also modify it, for example, a regular method can replace the contained object with another one.
 - Only the two strictest interpretations need to instantiate the contained object when establishing the relationship and destroy it when dismantling it.



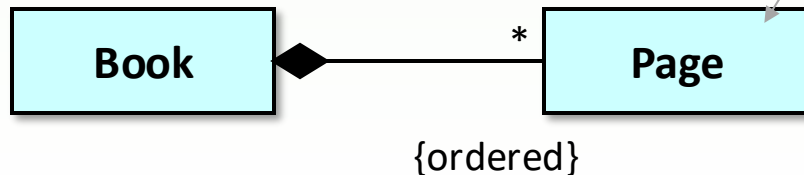
Aggregation vs Composition



Slides are parts of playgrounds, but a playground can still be a playground without a slide. The decision might be confusing because a slide can only belong to one playground at a time, even though it can be moved to another one and can be used on its own.

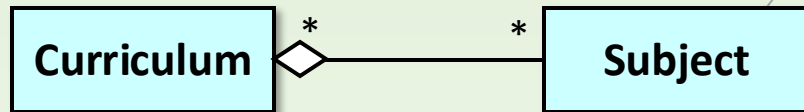


Walls are parts of a room, but the same wall can belong to two rooms at once. The decision might be confusing because without walls, there's no room.

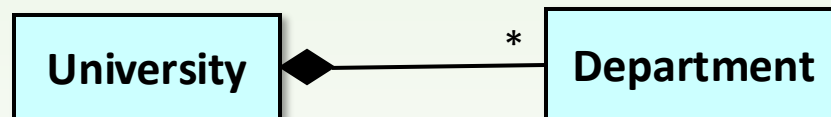


A book consists of a given number of pages, and its pages can only ever belong to that book. The decision isn't affected by whether a page of a book can be used on its own.

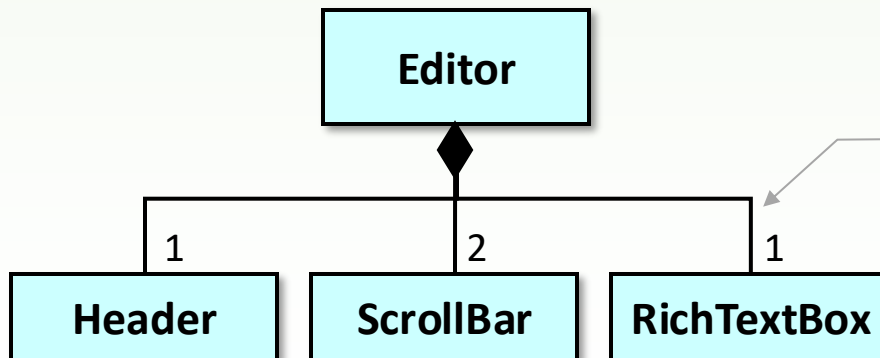
Aggregation vs Composition



While a curriculum doesn't exist without subjects, a subject can be part of multiple curricula at the same time (and doesn't even have to belong to a curriculum).



A university doesn't exist without a department (and a department doesn't exist without a university). It has departments from the time it's founded (but departments can be created later, and they can also be dissolved). A department can't be a department of more than one university at a time (it can't transfer to another university; if the university closes, then the department ceases to exist).

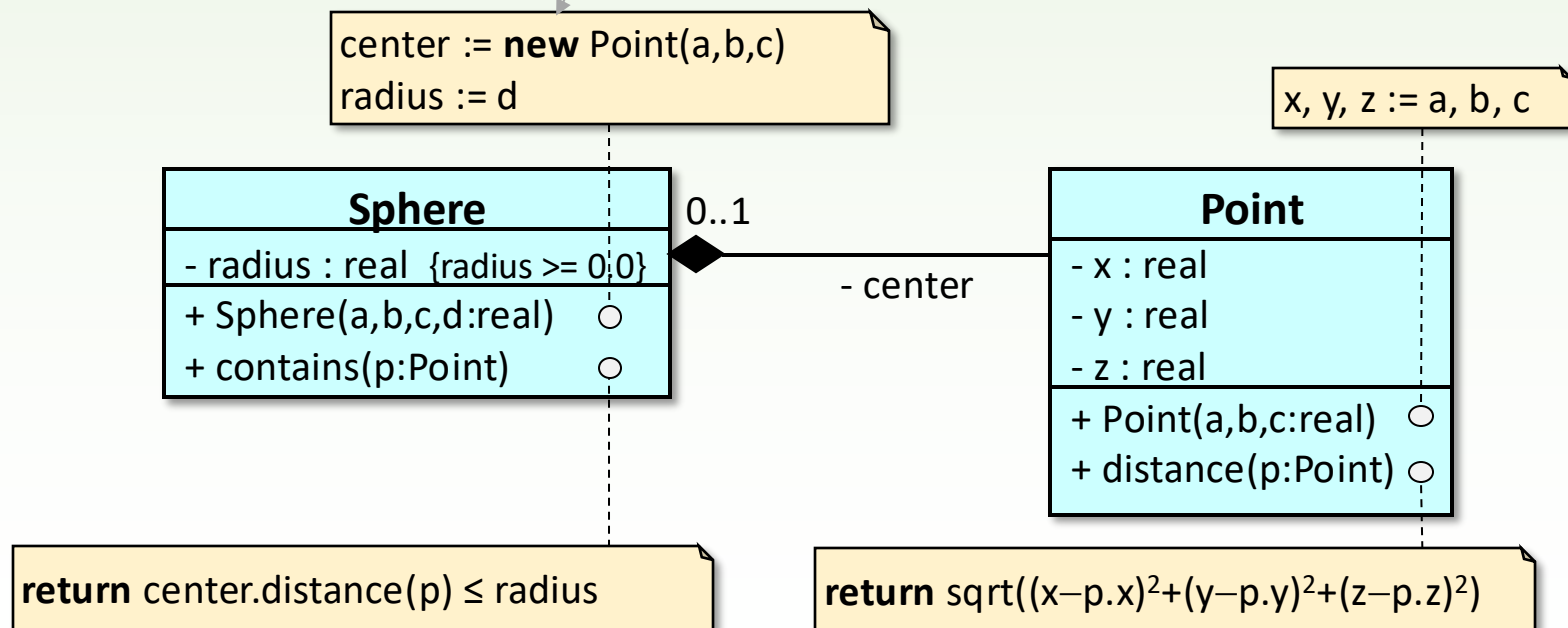


When the editor window is created, its parts are created at the same time: the header, scroll bars, and editor surface. These will then disappear when the window is closed.

Example: Point in sphere

- Let's model spatial points and spheres as part of solving a problem where we need to count how many of a given set of points are located within a given sphere.
- Idea: Let the center point be part of the sphere.

The composition relationship is built up by the constructor of the whole. Although it's not required in general, this is also where the contained object (the center point) is created.



Example: Point in a sphere

```
class Point {  
    private final double x, y, z;  
  
    public Point(double a, double b, double c) { x = a; y = b; z = c; }  
    public double distance( Point p) {  
        return Math.sqrt(Math.pow(x-p.x,2) + Math.pow(y-p.y,2) + Math.pow(z-p.z,2));  
    }  
}
```

The Java Math class methods allows you perform mathematical tasks on numbers.

```
class Sphere {  
    static class IllegalRadiusException extends RuntimeException {}  
    private final Point centre;  
    private final double radius;  
    public Sphere(Point c, double r) {  
        centre = c;  
        radius = r;  
        if (radius < 0.0) throw new IllegalRadiusException();  
    }  
    public boolean contains(Point p) {  
        return centre.distance(p) <= radius;  
    }  
}
```

Relationships of objects

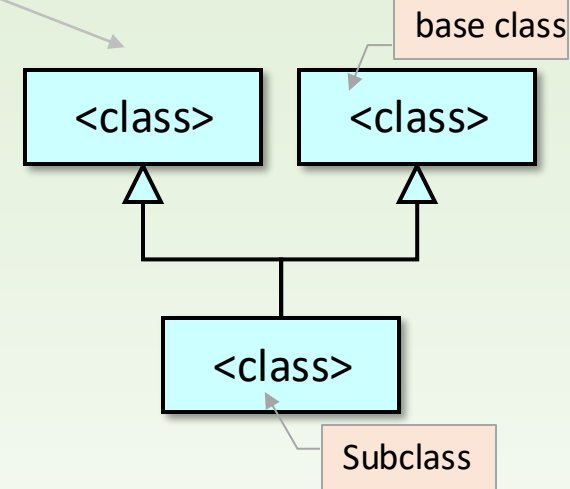
4th part
Inheritance

Gregorics Tibor and Pintér Balázs

Inheritance, subclassing

In Java, if we wanted to inherit from multiple base classes, all but one of them would have to be interfaces.

- ❑ Inheritance is the "is-a" relationship. For example, cat "is a" feline. You should always ask if "is-a" makes sense when using inheritance.
- ❑ The subclass inherits the fields and methods of the base class and can change them. The convenience of this alone is not enough to establish inheritance.



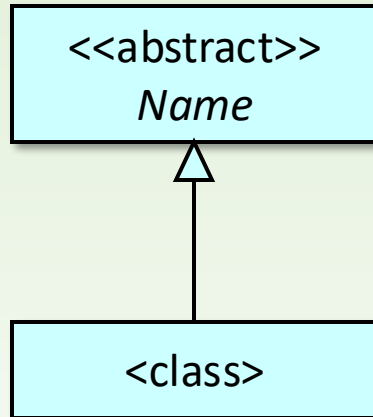
- We use inheritance during modeling for two main reasons:
 - **Generalization**: We create a parent class (**superclass**) that describes the common properties of existing, similar classes.
 - **Specialization**: We create a **subclass** from a class through inheritance.

5

Object-oriented languages support **inheritance**: classes can be derived from existing classes.

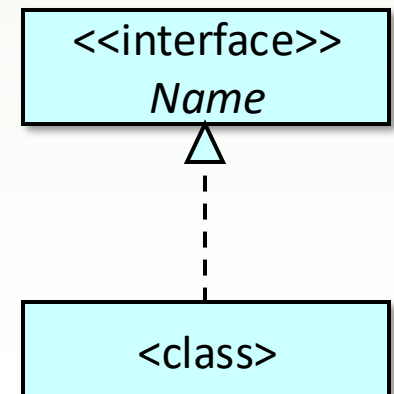
You can assign an object of a subclass to a variable of its parent class.

Abstract class, interface



- An **abstract class** is one from which we don't instantiate objects. It serves exclusively as a base class for inheritance. (In UML, we write the class name in italics, or we might indicate this separately with the <<abstract>> stereotype.)
- A class automatically becomes abstract if
 - its constructors aren't public, or
 - at least **one of its methods is abstract** (indicated by italics), meaning it doesn't have a body; this will need to be overridden in a subclass

- We call a purely abstract class an **interface**. It has no data members, and none of its methods have bodies.
- A concrete class derived from an interface has to implement all abstract methods: it has to **implement the interface**.



Inheritance and visibility

- A subclass inherits all members of its parent class, but it **can only refer to the parent class's public and protected members**. It can't access the private members directly; it can only access them indirectly through non-private methods inherited from the parent class.
- A subclass cannot reduce visibility of inherited methods but can increase it
- When overriding:
 - The overriding method must not be less accessible than the overridden method
 - The overriding method can have a more accessible modifier

Example: point from sphere

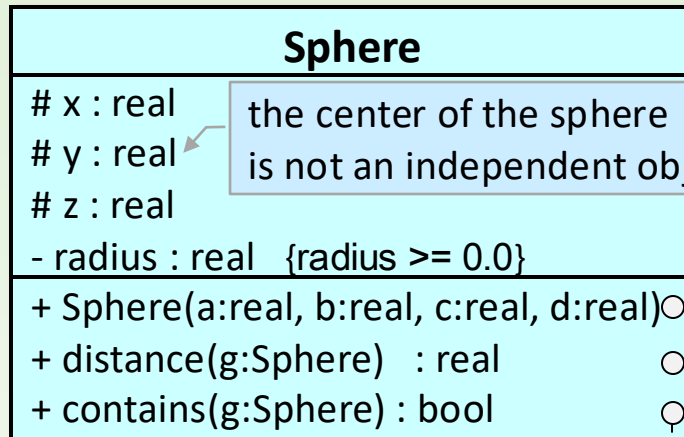
Single responsibility

O

Liskov's substitution

I

D



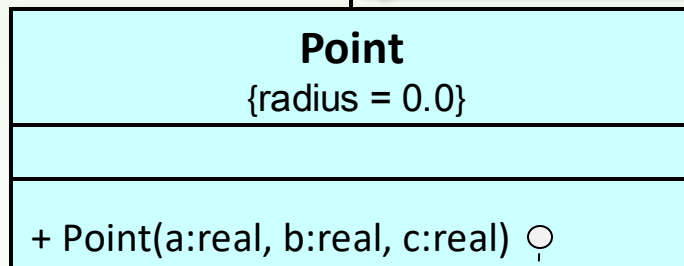
the center of the sphere
is not an independent object

if d<0 then error endif

x, y, z, radius := a, b, c, d

return sqrt((x-g.x)²+(y-g.y)²+(z-g.z)²) - this.radius -
g.radius

return distance(g) + 2 * g.radius ≤
0



Sphere(a, b, c, 0.0)

An overridden method, whether called on an object of the base class or a subclass, should operate in accordance with the expectations of the modeled task.

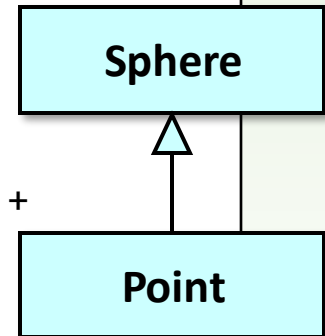
1. Because of inheritance, you can call the Sphere methods on a Point type object too:
d := p.distance(q) p, q : Point

2. A variable of the base class type can be assigned an object of its subclass (polymorphism), so the methods' Sphere-type parameter can also refer to a Point-type object:
l := a.contains(p) a : Sphere, p : Point

Java: Point from Sphere

```
Point p = new Point(0, 0, 0);
Sphere g = new Sphere(1, 1, 1, 1);
System.out.println("p.distance(p): " + p.distance(p));
System.out.println("g.Distance(p): " + g.distance(p));
System.out.println("p.distance(g): " + p.distance(g));
System.out.println("g.distance(g): " + g.distance(g));
System.out.println("g.contains(p): " + g.contains(p));
System.out.println("g.contains(g): " + g.contains(g));
```

```
class Sphere { protected
    static class IllegalRadiusException extends RuntimeException {}
    protected final double x, y, z;
    private final double radius;
    public Sphere(double a, double b, double c, double d) {
        x = a; y = b; z = c;
        radius = d;
        if (radius < 0.0) throw new IllegalRadiusException();
    }
    public double distance(Sphere g) {
        return Math.sqrt(Math.pow((x-g.x),2) + Math.pow((y-g.y),2) +
Math.pow((z-g.z),2))
        -radius - g.radius;
    }
    public boolean contains(Sphere g) {
        return distance(g) + 2 * g.radius <= 0;
    }
}
```



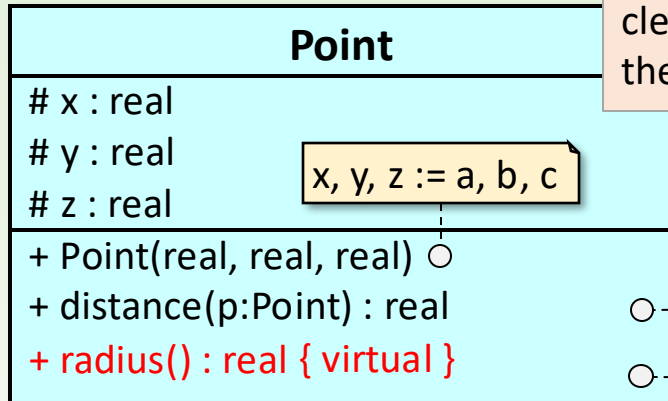
indicate that a class is
inheriting from a parent

is used to call a constructor
or methods from the parent class.

```
class Point extends Sphere {
    public Point(double a, double b, double c) { super(a, b, c, 0.0); }
}
```

Example: Sphere from Point

Whether p (or this) refers to a point or a sphere isn't clear at compile time, only at runtime. So, how does the compiler know which radius() to compile here?



x, y, z := a, b, c

~~return sqrt((x-p.x)²+(y-p.y)²+(z-p.z)²) - radius()~~ ~~p.radius()~~

~~return sqrt((x-p.x)²+(y-p.y)²+(z-p.z)²) - radius = p.radius~~

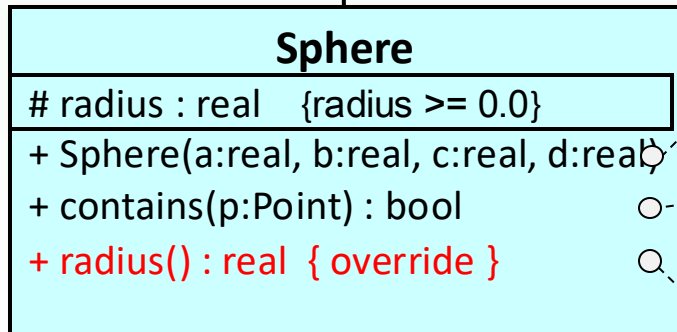
~~return sqrt((x-p.x)²+(y-p.y)²+(z-p.z)²)~~

return 0.0

If we call it on a sphere, it gives the wrong result: it violates the LSP.

A Point doesn't have a radius.

If a method overridden in subclasses is called on a base class type, then the version of the method that will run is the one belonging to the class of the object the method was called on.



if d<0 then error endif

Point(a, b, c); radius := d

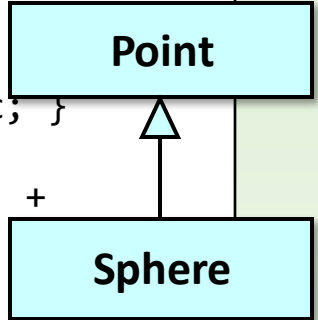
~~return distance(p) ≤ radius~~

return distance(p) + 2* p.radius() ≤ 0

return radius

Java: Sphere from Point

```
class Point {  
    private final double x, y, z;  
    public Point(double a, double b, double c) { x = a; y = b; z = c; }  
    public double distance(Point p) {  
        return Math.sqrt(Math.pow(x - p.x, 2) + Math.pow(y - p.y, 2) +  
Math.pow(z - p.z, 2) ) - radius() - p.radius();  
    }  
    public double radius() { return 0.0; }  
}
```



this.radius()

```
class Sphere extends Point {  
    static class IllegalRadiusException extends RuntimeException {}  
  
    private final double radius;  
  
    public Sphere(double a, double b, double c, double d) {  
        super(a, b, c); radius = d;  
        if (radius < 0.0) throw new IllegalRadiusException();  
    }  
  
    public boolean contains(Point p) {  
        return distance(p) + 2 * p.radius() <= 0;  
    }  
  
    @Override  
    public double radius() {  
        return radius;  
    }  
}
```

overridden method

```
Point p = new Point(0, 0, 0);  
Sphere g = new Sphere(1, 1, 1, 1);  
System.out.println("p.distance(p): " + p.distance(p));  
System.out.println("g.Distance(p): " + g.distance(p));  
System.out.println("p.distance(g): " + p.distance(g));  
System.out.println("g.distance(g): " + g.distance(g));  
System.out.println("g.contains(p): " + g.contains(p));  
System.out.println("g.contains(g): " + g.contains(g));
```

Dynamic subtype polymorphism

- If we **override** a method of a base class in its subclasses, that method will have multiple forms, which is called **polymorphism**. Which version gets executed depends on which subclass (or maybe the base class) is the type of the object at **runtime**. This is also called **dynamic binding** or **late binding**.

6

A key feature of object-oriented languages is **dynamic subtype polymorphism**